

Spring AI 개발, Harness로 10배 빠르게

| Claude Code 방법론이 바꾸는 AI 개발의 새로운 패러다임

 발표자 | 황민호 수석 (카카오 AI Studio FDE)

 Spring Camp 2026



발표자 소개 · AI 개발 여정



WIPRO (2009)

SQL BNF 문법 기반 자동 SQL 생성기로
DBMS 테스트



KISTI (2010)

문장 패턴 분석을 통한
온톨로지 인스턴스 구축



DAUM (2013)

클라우드 서비스 · 검색 서비스 경험



KAKAO (현재)

광고 플랫폼, 오픈소스, Tech Talk,
AI 교육 자문, AI Native 전략, AI Studio-FDE



다양한 플랫폼과
AI 경험이
오늘의 이 발표로
이어졌습니다!



일단 그냥 시작해봅시다

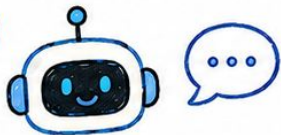
AI 프로젝트 시작 프롬프트 예시

1



Spring AI 최신 버전
백엔드 프로젝트 시작

2



GPT-5.5 챗봇
OpenAI 기반 API

3



테스트용 모던 페이지
API 확인용 화면

4



Tailwind CSS + shadcn
깔끔한 UI 스타일

“

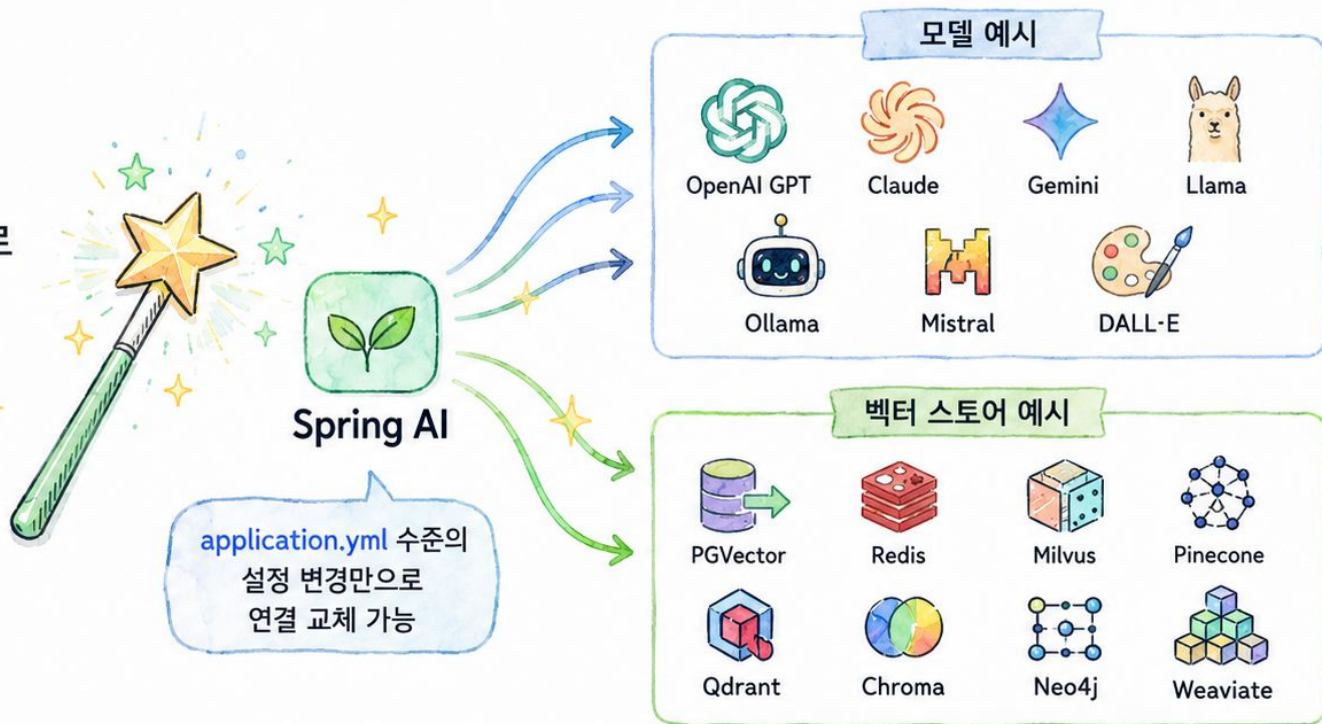
Spring AI 최신 버전으로 백엔드 프로젝트를 시작합니다.
OpenAI GPT-5.5 기반 챗봇 시스템을 만들고,
이 API를 테스트할 모던 스타일의 페이지가 필요합니다.
Tailwind CSS + shadcn 적용.

”

핵심은 '일단 시작해보기' — 요구사항을 자연어로 한 번에 전달

Spring Boot의 마법이 거대한 AI 생태계를 만났을 때 ✨

하나의 라이브러리 추상화로
다양한 AI 모델과
벡터 스토어를 연결



Spring AI가 우리에게 한 달콤한 약속

코드 한 줄 변경 없이
application.yml
수정만으로 모델 교체

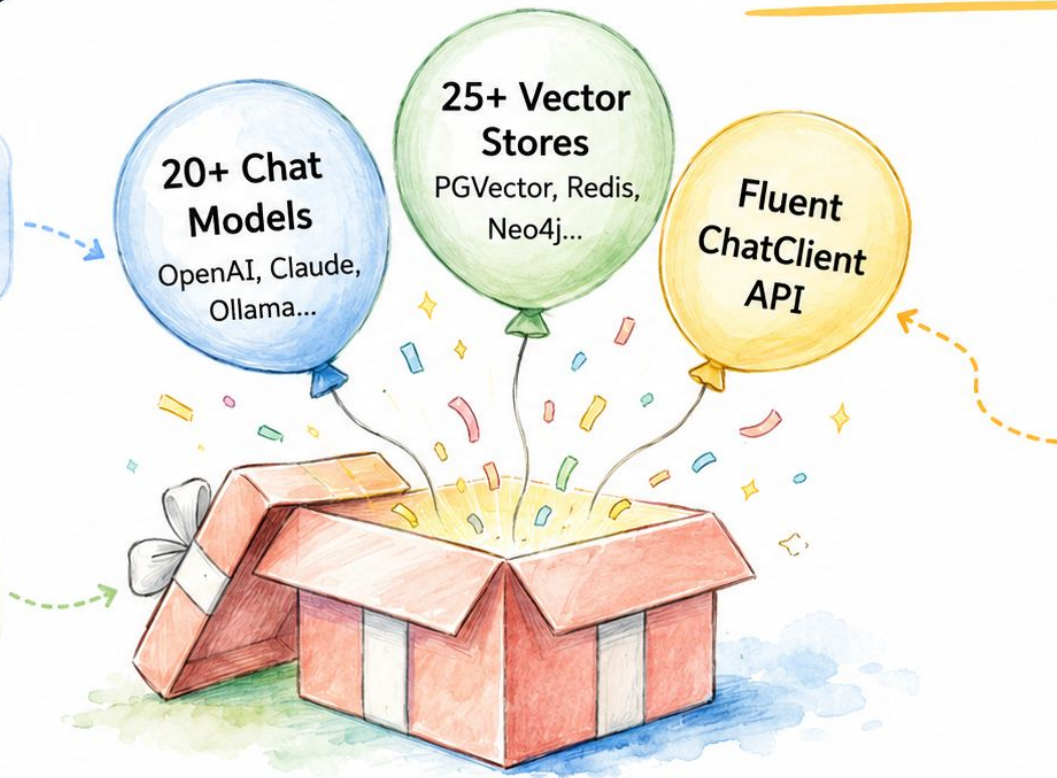
20+ Chat
Models
OpenAI, Claude,
Ollama...

25+ Vector
Stores
PGVector, Redis,
Neo4j...

Fluent
ChatClient
API

라이브러리 추가하듯
자연스러운 AI 통합

우리가 사랑하는
익숙한 Spring
생태계 그대로



하지만 우리가 마주한 현실은...



1. API 불안정성

모델 버전 변경 후
Tool Calling이
조용히 실패



2. 끝없는 보일러플레이트

간단한 RAG 하나에도
여러 클래스와
설정 필요



3. 테스트 지옥

비결정적 응답 앞에서
Mocking과
검증이 어려움



견고해 보이던 아키텍처 내부에서 발생하는 런타임 충돌



RAG + Tool Calling 충돌

RetrievalAugmentationAdvisor와
ToolCallbacks를 함께 쓰면,
검색 컨텍스트 의존이 과도해지며
복합 에이전트 구성이
흔들릴 수 있다.



실전 이슈로 드러난 구조적 한계



스트리밍 데이터 유실

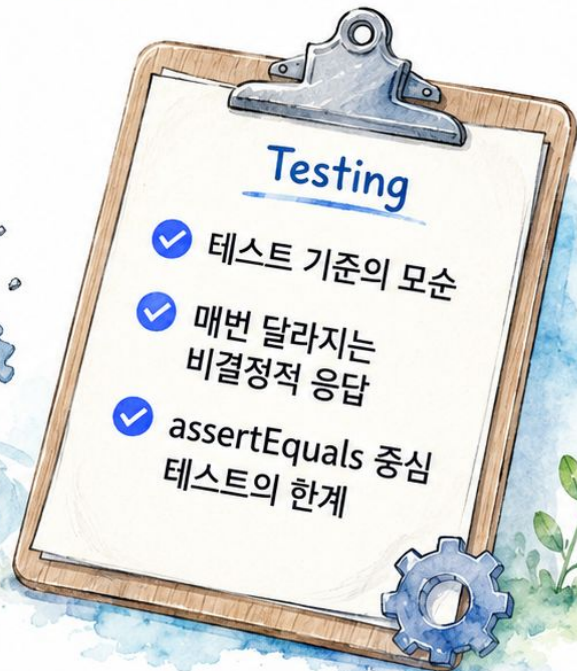
Flux 기반 스트리밍에서
메시지 재구성 과정 중
toolCalls 정보가 누락될 수 있어
응답 안정성이 떨어진다.

모든 문제의 기저에 깔린 단 하나의 원인



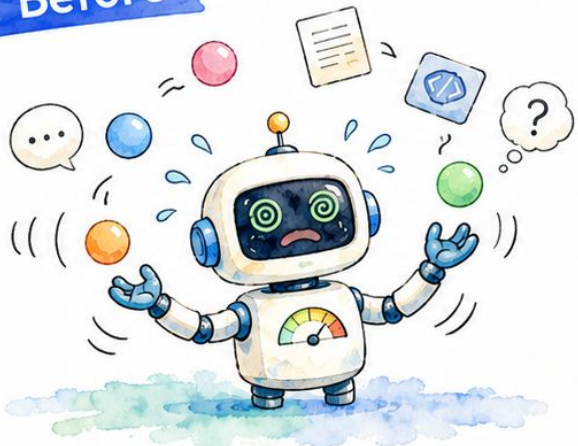
Spring AI 컴포넌트의 조합과 검증을 **개발자 혼자 모두 떠안는 구조**가 문제의 핵심이다.

결코 자동화되지 않는 테스트와 프롬프트 관리의 늪



혼자 고군분투하는 대신, 나만의 **AI 개발 팀**을 고용할 수 있다면?

Before



- ✓ 단일 대화창에 텍스트 쏟아붓기
- ✓ 지식 파편화 & 조합의 복잡성

After



- ✓ 조직화된 전문 에이전트 팀 분업
- ✓ Claude Code Harness 시스템



Claude Code Harness의 3대 핵심 구성 요소



역할 · 재사용 · 조율로 AI 팀을 구조화합니다



핵심은 **역할**을 나누고, 반복을 **스킬**로 만들고, 전체를 **오케스트레이션**하는 것



Spring 개발자에게 가장 잘 맞는 멘탈 모델



Spring Framework 세계



@Component
역할 기반 컴포넌트



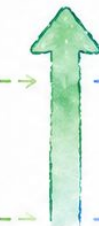
@Configuration + @Bean
선언적 기능 단위



@SpringBootApplication
전체 서비스 조율



ApplicationEventPublisher
비동기 메시징



멘탈 모델
매핑



Claude Code Harness 세계



Agent
역할 기반 AI 전문가



Skill
재사용 가능한 워크플로우



Orchestrator
에이전트 조율



SendMessage
A2A 통신



Spring 감각 그대로, AI 팀을 설계한다



당신의 새로운 AI 팀원들

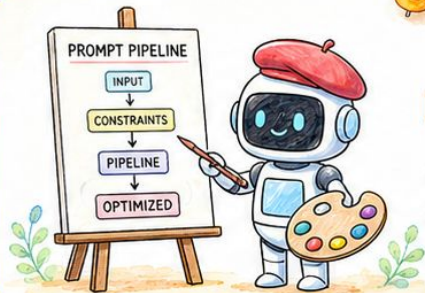


역할을 나누면 복잡도가 줄어듭니다



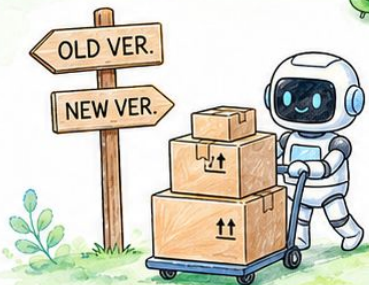
Integration Architect

RAG와 Tool Calling 충돌을
예방하는 아키텍처 담당



Prompt Designer

프롬프트 파이프라인을
설계하고 제약을 우회



Migration Specialist

버전 변화와 Breaking Change를
감지해 이관 지원



Test Engineer

Mock 전략과 통합 테스트로
AI 품질 검증

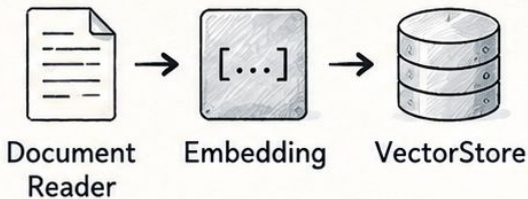


시나리오 1. RAG 파이프라인 수동 구축의 종말



더 빠르게, 더 똑똑하게. 팀이 함께 만드는 RAG 파이프라인

수동 구축



4~8시간

공식 문서를 뒤져가며
컴포넌트를 수동 조립

Harness 팀 협업



1~2시간



문서 읽기



PGVector 설정



테스트 코드 작성



구축 시간 75% 단축



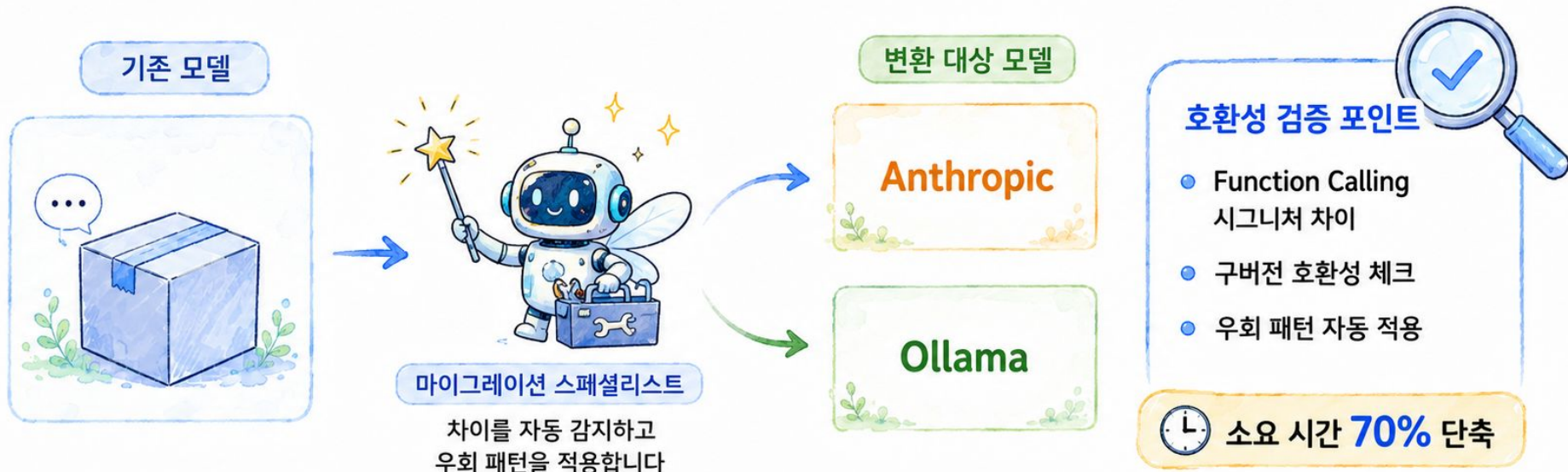
혼자 순서대로 하던 일을, **팀이 병렬로** 끝낸다



시나리오 2. 멀티모델 전환과 호환성 검증



에이전트가 자동 감지하고 우회 패턴을 적용하여 안전하게 전환합니다



application.yml 변경만으로 끝나지 않는 차이를 에이전트가 메웁니다

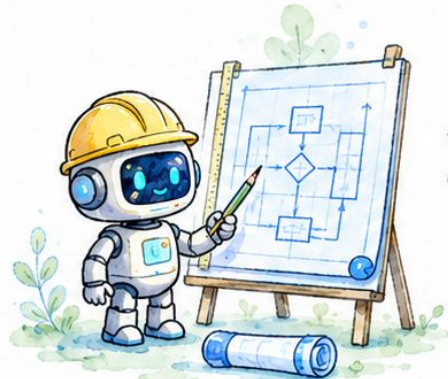




시나리오 3. 복잡한 워크플로우 설계와 충돌 예방



RetrievalAugmentationAdvisor (RAG)



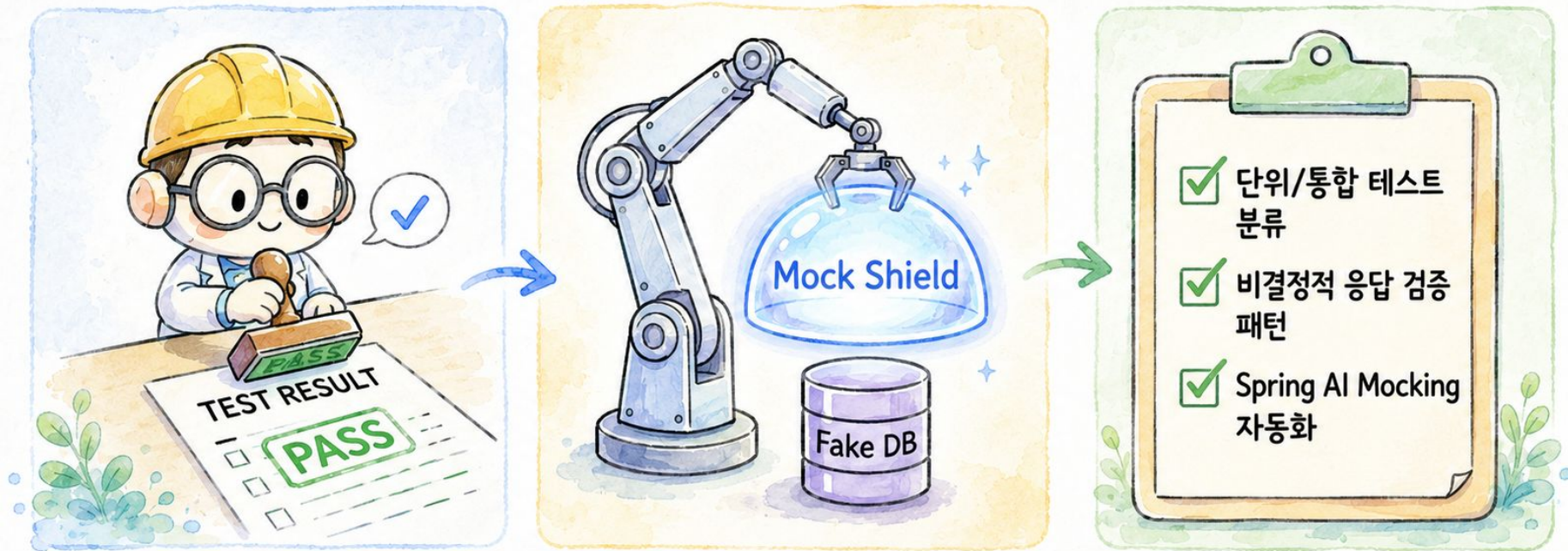
ToolCallAdvisor



RAG 컨텍스트와 Tool Calling을 무작정 섞지 않고,
안전하게 분리 설계합니다



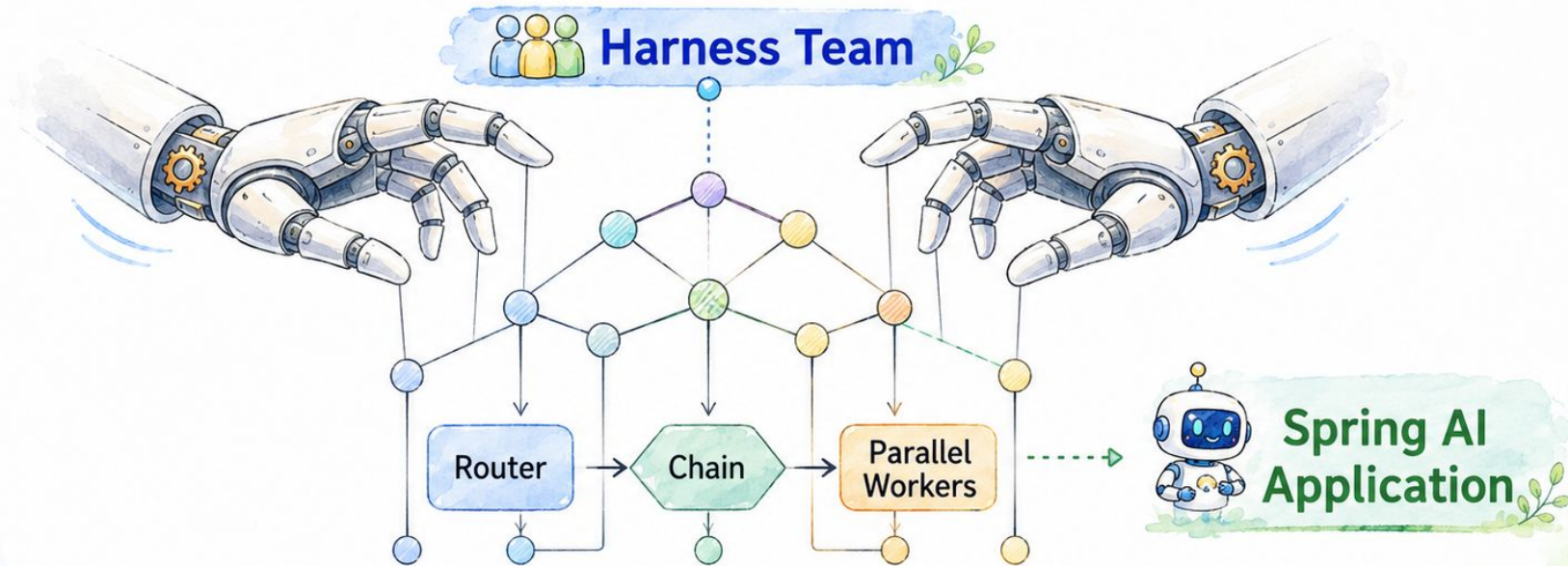
시나리오 4. 가장 포기하기 쉬운 AI 테스트의 자동화



매번 답변이 달라도, **테스트 전략은 자동화**할 수 있습니다



메타적 시너지: Agentic 앱을 Agentic하게 개발하다



개발 팀의 구조가, 우리가 만들 앱의 구조를 그대로 리허설한다



다음 단계: MCP (Model Context Protocol) + A2A (Agent-to-Agent) 기반 통신 아키텍처로 확장



정량적 단축을 넘어선 개발 경험의 진화



RAG 파이프라인 구축
— 시간 **75%** 절감



멀티모델 전환 & 검증
— 시간 **70%** 절감



AI 테스트 코드 작성
— 시간 **70%** 절감



혼자 다 하기 → **팀으로 병렬 협업**



매번 처음부터 → **스킬로 영구 재사용**



범용 지식 → **전문가 팀의 분리된 컨텍스트**



속도만 빨라진 것이 아니라, **개발 방식 자체가 바뀝니다**





Agentic 앱은, **Agentic**한 방식으로 개발해야 합니다



Agentic App

우리가 만들 서비스



Chain



Parallel



Orchestrator

Harness AI Team

우리가 일하는 방식



Agents



Skills



Orchestrator
workflows



앱의 구조와 개발 팀의 구조가 서로를 강화합니다



Harness로 팀을 구성하는 과정 자체가 **차세대 AI 시스템의 리허설**입니다

Spring AI Harness로 만드는 AI 개발팀

새로운 언어 없이, 역할 분리와 병렬 실행으로 빠르게 구축

★ 핵심 포인트

- ✓ 복잡한 인프라보다 협업 구조가 중요
- ✓ Spring AI가 챗봇 백엔드 구현을 단순화
- ✓ 에이전트 분업으로 개발·검증을 병렬화



1



폴더와 역할 정의

.claude/ 또는
에이전트 폴더 구성

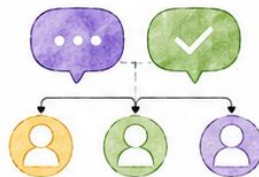
2



Markdown으로
스킬 선언

역할·입출력·책임을
짧게 명시

3



대화형 호출로 협업

필요한 스킬을 불러
작업 연결

★ 핵심 메시지: 마크다운 몇 줄로 AI 팀의 작업 흐름을 설계할 수 있다

Spring AI + OpenAI 챗봇 구축 흐름

CLI에서 테스트 페이지까지, 구현 단계를 한 화면에

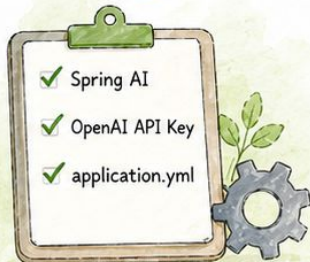
1



프로젝트 생성

/harness:harness로
최신 Spring AI 백엔드 생성

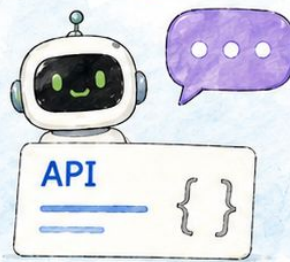
2



백엔드 설정

Spring AI 의존성
OpenAI API Key
application.yml

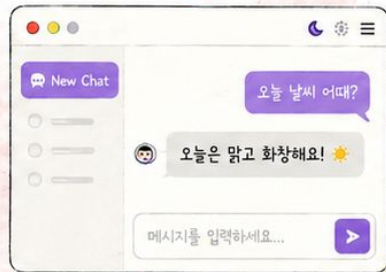
3



챗봇 API 구성

ChatClient
POST /api/chat
GET /api/health

4



테스트 UI

React + Tailwind + shadcn/ui
대화 기록 · Markdown · 공유



관련 인사이트



ChatClient = 동기·스트리밍 지원



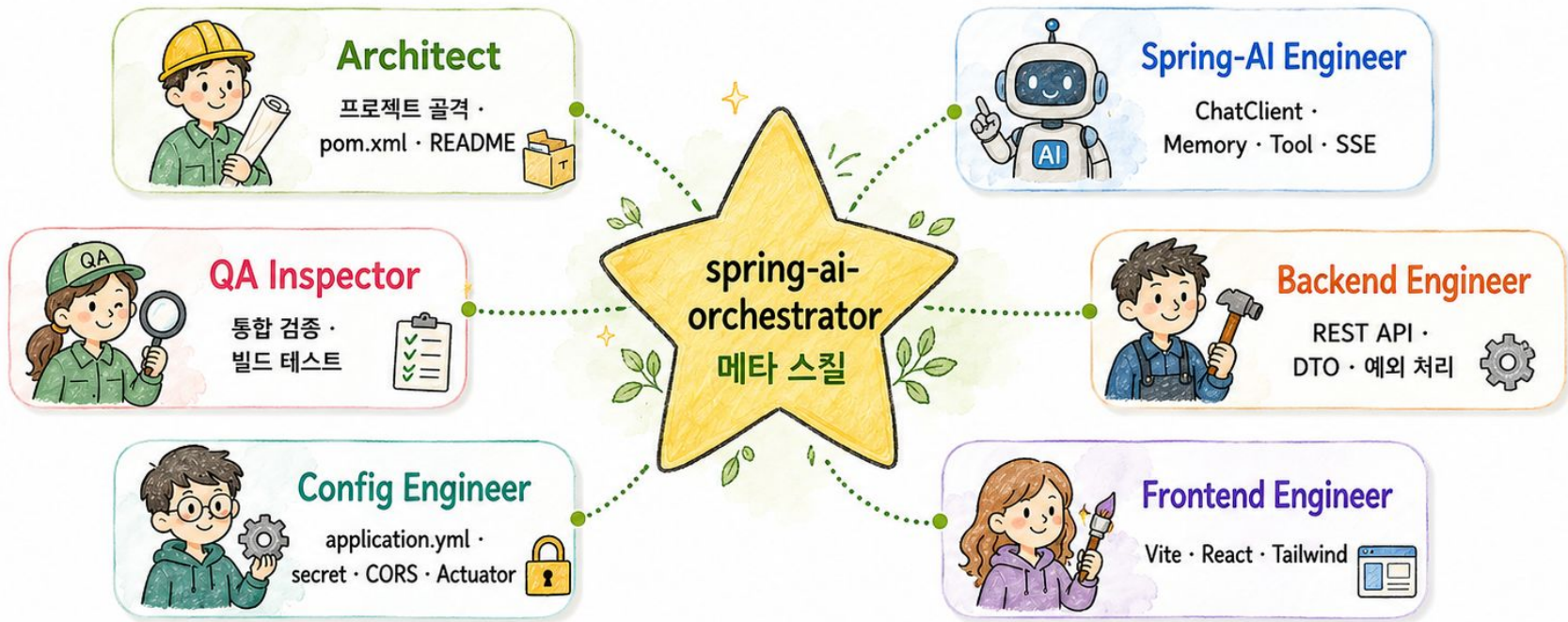
Tool Calling = 외부 기능 연결



Memory = 대화 맥락 유지

에이전트 & 스킬 맵

Orchestrator가 역할별 스킬을 연결해 병렬 협업을 만든다





Phase별 빌드 파이프라인



빠대 구축 → 병렬 구현 → 검증 → Frontend 확장



Phase 1 · 골격 잡기

spring-project-init



📁 pom.xml · 패키지 폴더 · README

Phase 2 · 병렬 구현



spring-ai-chat

ChatClient · Memory ·
Tool · SSE



spring-rest-api

REST · DTO · Exception



spring-config

application.yml · secret · CORS

★ 속도의 핵심: 병렬화

Phase 3 · 검증

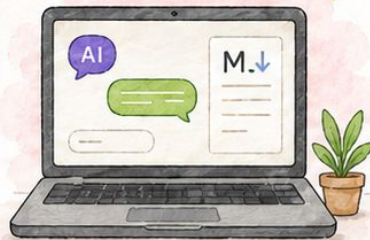
integration-qa



mvn test
11/11 PASS

확장 · Frontend

frontend-test-page



채팅 UI · Markdown · 공유

실무 적용 설계 포인트

Spring AI 공식 기능과 Harness 구조를 연결하는 운영 관점



1 표준 인터페이스

모델별 차이를 추상화해
교체 비용을 낮춤



2 ChatClient 중심 설계

동기 · 스트리밍 응답을
같은 흐름에서 관리



3 Tool + Memory

업무 기능 호출과
대화 맥락 유지를 결합



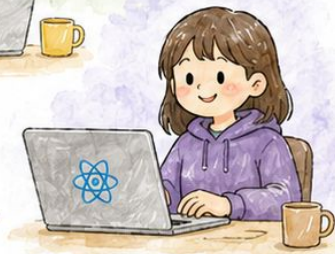
4 운영 준비

설정 · 보안 · 테스트 ·
관측성을 초기부터 포함

Backend Team



Frontend Team



추천 실행 흐름



Orchestrator



역할별 skill call



QA verify



Frontend delivery